From simple to complex: A sequential method for enhancing time series forecasting with deep learning

M. J. JIMÉNEZ-NAVARRO[*], *Department of Computer Science, University of Seville, 41012 ES-Seville, Spain.*

M. MARTÍNEZ-BALLESTEROS[**], *Department of Computer Science, University of Seville, 41012 ES-Seville, Spain.*

F. MARTÍNEZ-ÁLVAREZ[†], *Data Science and Big Data Lab, Pablo de Olavide University, 41013 ES-Seville, Spain.*

A. TRONCOSO[††], *Data Science and Big Data Lab, Pablo de Olavide University, 41013 ES-Seville, Spain.*

G. ASENCIO-CORTÉS[§], *Data Science and Big Data Lab, Pablo de Olavide University, 41013 ES-Seville, Spain.*

Abstract

Time series forecasting is a well-known deep learning application field in which previous data are used to predict the future behavior of the series. Recently, several deep learning approaches have been proposed in which several nonlinear functions are applied to the input to obtain the output. In this paper, we introduce a novel method to improve the performance of deep learning models in time series forecasting. This method divides the model into hierarchies or levels from simpler to more complex ones. Simpler levels handle smoothed versions of the input, whereas the most complex level processes the original time series. This method follows the human learning process where general/simpler tasks are performed first, and afterward, more precise/harder ones are accomplished. Our proposed methodology has been applied to the LSTM architecture, showing remarkable performance in various time series. In addition, a comparison is reported including a standard LSTM and novel methods such as DeepAR, Temporal Fusion Transformer, NBEATS and Echo State Network.

Keywords: time series, forecasting, deep learning; univariate; multihorizon

[*]E-mail: mjimenez3@us.es
[**]E-mail: mariamartinez@us.es
[†]E-mail: fmaralv@upo.es
[††]E-mail: atrolor@upo.es
[§]E-mail: guaasecor@upo.es

Vol. 32, No. 6, © The Author(s) 2024. Published by Oxford University Press.
This is an Open Access article distributed under the terms of the Creative Commons Attribution Non-Commercial License (http://creativecommons.org/licenses/by-nc/4.0/), which permits non-commercial re-use, distribution, and reproduction in any medium, provided the original work is properly cited. For commercial re-use, please contact reprints@oup.com for reprints and translation rights for reprints. All other permissions can be obtained through our RightsLink service via the Permissions link on the article page on our site-for further information please contact journals.permissions@oup.com.
Advance Access published 14 May 2024 https://doi.org/10.1093/jigpal/jzae030

Downloaded from https://academic.oup.com/jigpal/article/32/6/986/7670726 by Universidad de Burgos user on 11 September 2025

1 Introduction

Time series forecasting is a crucial task with several applications like electricity demand, weather, pollution, etc. Time series is defined as a set of data ordered over time. To forecast future behavior, known information from the past is used because, due to the causal nature of the time series, this information helps to determine the future. When several time series including a set of features are considered, the problem is known as multivariate, whereas a single time series including one feature leads to a univariate problem. We can predict either only one-step ahead or multiple steps, known as multistep forecasting [7].

In this paper, we address multistep forecasting in univariate time series due to its importance in many practical scenarios such as traffic management, electricity consumption or store sales [23]. In many of these scenarios, we need to look at the evolution over multiple time steps to perform decisions and strategies to make decisions [28].

Deep learning has been used increasingly in multistep time series forecasting [30], demonstrating great improvements over the traditional Box–Jenkins and machine learning models. Deep learning models perform the forecast feeding past events, learning the features, and obtaining future behavior from these features. In these cases, the model learns some unknown relationship between the inputs and the outputs, converting them into black boxes.

In this paper, we try to shed some light on models that assign an additional task to each layer, or group of layers, in the neural network. For that purpose, we make the assumption that, like humans, neural nets can learn faster, starting from simpler tasks and increasing the difficulty incrementally. For example, if you wanted to learn to ride a motorcycle, it would be faster if you already knew how to ride a bike since you have learned concepts such as equilibrium, brake, circulation, etc. Learning these concepts directly from a more complex task can be time-consuming as there are more things to consider compared with the easiest task. Even if the concepts are learned from just the complex task, it could be biased to that task and suboptimal. Following the analogy with the motorbike, if you can ride the motorbike and the bike, then probably you will have a less unbiased sense of equilibrium. This unbiased sense is because, e.g. you cannot associate it with just the weight or shape of the motorcycle.

In this context, we propose a novel method to improve the neural networks performance for time series forecasting, hereinafter called Hierarchical Learning Network (HLNet). For that purpose, we divide the neural network into a hierarchy specifying a simpler input to the firsts layers and a more complex input to the following layers. Note that we consider a simple hierarchy of tasks formed as a sequence.

Simplification is obtained by applying a smoothing process to the input. The first layers transfer their knowledge to all the following layers, with which the following layers can take advantage of this knowledge and learn complex outcomes faster. Our proposal uses LSTM layers as basic blocks as recent literature shows the best results in most of datasets [17].

The main contributions of this paper are summarized as follows:

- A novel methodology applied to a LSTM keeping a similar or better efficacy among several datasets while improving the efficiency.
- The proposal of an interpretable mechanism applied to a model that includes some keys of the learned behavior to each layer.
- The comparison of our proposal with LSTM [11], NBEATS [24], DeepAR [26], Temporal Fusion Transformers (TFT) [20] and Echo State Network (ESN) [12] using 13 different datasets from different application fields. DeepAR and ESN were selected due to it uses

Downloaded from https://academic.oup.com/jigpal/article/32/6/986/7670726 by Universidad de Burgos user on 11 September 2025

recurrent layers, while NBEATS and TFT were selected because it obtained remarkable results in several datasets.

The remainder of this paper is structured as follows. In Section 2, we describe the work related to this paper focusing on the main differences. Section 3 we describe the methodology applied to this paper. Section 4 shows the experimentation carried out with a wide set of datasets and a scalability analysis comparing HLNet with the best model with best results in the experimentation. Finally, we discuss the conclusions of this work and future directions.

2 Related works

Previous related work consider the use of multi smoothing or multiresolution time series in order to improve the performance of the models. Usually these methods use some model to learn each representation and then join each one. In some cases, these methods are inefficient because they need to train as many models as representations and consider the learning of each representation independently. Another type of methods change the training algorithm using some heuristic in order to learn before from simpler instances.

Kourentzes and Petropoulos [15] propose an extension of MAPA [14] and tested stock keeping unit demand. Their method uses several non overlapping representations of the data and fit an arbitrary model for each one, then the models predictions are averaged by an unweighted mean or median. Although, this work makes use of several smoothing methods to make more accurate and robust predictions, our proposed method uses them differently. In our method, we do not isolate the models and let them leverage automatically their knowledge to improve the predictions and all the representations are trained jointly improving the training performance.

Afolabi *et al.* [1] propose a method in which a noise elimination process based on empirical mode decomposition is used in original time series generating what they call meta-info, then the original and the meta-info is fed into a model to get the predictions. The authors tested their method on the NARX networks [21] and LSTM with good results. Our proposed method do not focus in noise elimination, in the simplification process we do not worry about losing information but obtain some insights about the time series. Furthermore, the simplification lines up with the model architecture, and we do not only input the simplified versions as an attribute of the input.

Wu *et al.* [33] proposed the RESTful framework in order to improve the forecasting performance using the multi-resolution patterns inside the time series. The authors proposed a process of two steps, in the first step the model uses an GRU network to extract the hidden state of each time series and each resolution, then at the second step a convolutional layer is used to extract the pattern of each hidden state and obtain the predictions. Our method do not just lies in the multi-resolution representations but also in the smooth representation of the time series to extract the useful patterns. Furthermore, the proposed method is an one-step process in which the extraction of features and the final predictions are produced in a single model. Finally, the multiple representations of the time series are not isolated as we previously mentioned.

Bahrpeyma *et al.* [3] used multiple representations of time series and trained a recurrent neural network for each one. The recurrent neural networks predict the multiple horizons and finally use a weighted average based in the model performance. As in previous work, this method presents the issue of model isolation and performance.

Salinat *et al.* [26] improved an LSTM model by adding a probabilistic forecasting in which the forecasting corresponds to an range instead of a single value. The range is obtained using a likelihood model that determines the parameters of the predicted distribution, such as a normal

Downloaded from https://academic.oup.com/jigpal/article/32/6/986/7670726 by Universidad de Burgos user on 11 September 2025

distribution. Although this model is effective, presents the same problems as an usual LSTM in terms of efficiency. Additionally, the probabilistic forecasting is out of the scope of this work.

B. N. Oreshkin *et al.* [24] propose a novel interpretable architecture called NBEATS. The key idea is the inclusion of backward and forward residual links. The backwards reduce the complexity of the input time series, while the forwards are a partial prediction. The final forecasting sums up all the partial predictions. Although the architecture is very efficient, its structure is based on fully connected layers, which may perform poorly in time series datasets. Additionally, interpretability is based on a trend-seasonality decomposition that imposes a strong inductive bias on the model.

A similar idea to our proposal is the curriculum learning [13]. In this approach, the complexity of the task is increased during the training ordering the data by some criteria. Koenecke and Gajewar [13] applied the curriculum learning to financial time series and obtained good results. They used the baby steps algorithm and the seasonal decomposition as the complexity ordering criteria. In our method, we focus on changing the architecture more than the training algorithm, without making assumptions about the complexity into segments of the time series.

Another idea similar to our proposal is multitask learning. In this type of learning, the models try predicting several outputs for each task using a shared model for all tasks. Jinliang Deng *et al.* [5], propose a multitask time series forecasting method that adds a Multi-View Multi-Task (MVMT) block to residual blocks in the WaveNet architecture [32]. Although similar, the learning process is not the same. HLNet would be more similar to auxiliary learning [19] as we are interested in optimizing the final prediction task using other tasks. However, to our knowledge, there is no multistep time series forecasting architecture for hierarchical auxiliary tasks.

Recent architectures use the novel proposed self-attention mechanism [18, 38, 39]. However, this mechanism use a sparse approach to compensate the issues presented by the self-attention mechanism like quadratic time complexity or high memory consumption. Additionally, recent studies [36] discuss the effectiveness of this type of methods being far from substitute previous architectures.

3 Methodology

Traditional deep learning networks stack a set of layers. Some studies seems to indicate that some type of layers, like convolutional[35], learn more abstract low-level representations. Then, the following layers use this information to create higher-level representations and, finally, generate the prediction.

In this paper, we introduce a novel methodology applied to neural networks that proposes a new architectural design. In our proposal, every group has an input and two outputs. The input is a simplified version of the original input, the first output is the knowledge learned in the group and the second output includes a simplified prediction. Each group of layers has a harder input/output than the previous groups. Thus, the first group of layers has the easiest objective and, as we process the following layers, more complex objectives are assigned.

To understand the proposed architecture, we must specify the concepts of simple and complex objectives. First, we define an input sequence as a multivariate time series $S_{m,w}(t)$, which stands for m univariate time series of length w. That is, $S_{m,w}(t) = \{S_1(t), S_2(t), \ldots, S_m(t)\}$, where every $S_i(t)$ identifies a single time series of length w.

Figure 1 illustrates an input multivariate time series $S_{10,1000}(t)$ with $w = 1000$ and $m = 10$.

The model simplifies the input sequence for each group i generating a sequence $\hat{S}^i_{m,w'}(t)$, which is simpler than $S_{m,w}(t)$. The definition of a simpler input sequence in this paper corresponds to a smoothed version of the input sequence. Smoothing helps us identify patterns in time series, such as trend, and obtain an input sequence with less noise and variation.

Downloaded from https://academic.oup.com/jigpal/article/32/6/986/7670726 by Universidad de Burgos user on 11 September 2025

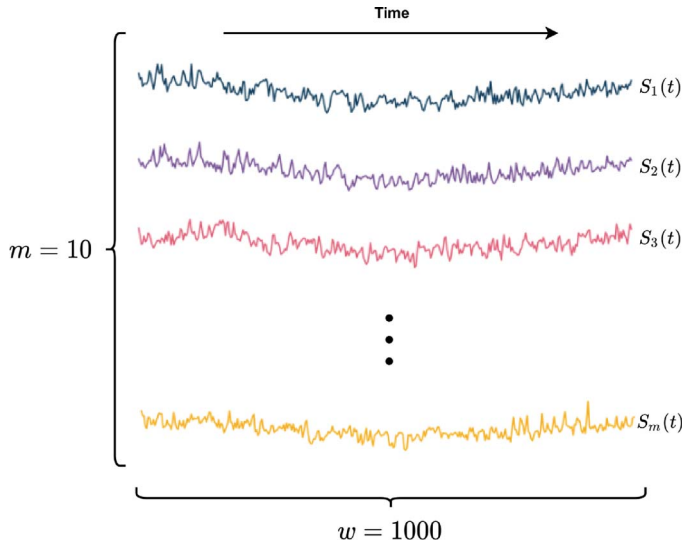


FIGURE 1. Input sequence $S_{10,1000}(t)$ composed of 10 time series with 1000 samples each.

The moving average has been used as the smoothing method. This method receives an hyperparameter (σ) that control the smoothness applied to the input of each group. The parameter $\sigma_i \in \mathbb{N}$, hereinafter smooth factor, specifies the window size on the moving average for each group i. The larger values of σ_i, the greater the smoothness, and also the degree of simplification. Therefore, the first groups have higher values of σ than the following groups. Note that $\sigma = 1$ for the last group of layers, keeping the input unmodified.

Figure 2 shows the HLNet architecture. Each group (G_i) starts with the *SmoothLayer* providing their respective σ_i generating $\hat{S}^i_{m,w'}(t)$, where w' means that the time series length can vary after the smoothing process. The LSTM layer receives $\hat{S}^i_{m,w'}(t)$ and obtains the first output: the hidden state H_i, which encodes the learned representation of the entire sequence. Then, the current hidden state is concatenated with previous hidden states. This is a critical step as, at this point, the previous groups share the knowledge learned from simpler representations with subsequent groups.

Being G_i the function for the group i that outputs its hidden state H_i and f_i the *FeedForward* layer from group i, the output for each output is defined as a function of the current group concatenated with all previous hidden states:

$$O^i_{1,\ldots,h} = f_i(G_i(S_{m,w}(t)) + \ldots + G_1(S_{m,w}(t)))$$

Note that the arrow that goes from the concatenation of the hidden states to the next group in Figure 2 is dashed as the gradients of this connection are not considered in the backpropagation algorithm. Not considering this connection isolates the learning process for each group, which prevents other groups from modifying the current group at backpropagation. Finally, the concatenated hidden state is fed into the *FeedForward* layer of group i, which outputs the predictions for each horizon of size h [8]. Each group i generates the second output $O^i_{1,\ldots,h}$ so, at the end, we have as many outputs as groups. However, only the final group has the real prediction.

Downloaded from https://academic.oup.com/jigpal/article/32/6/986/7670726 by Universidad de Burgos user on 11 September 2025

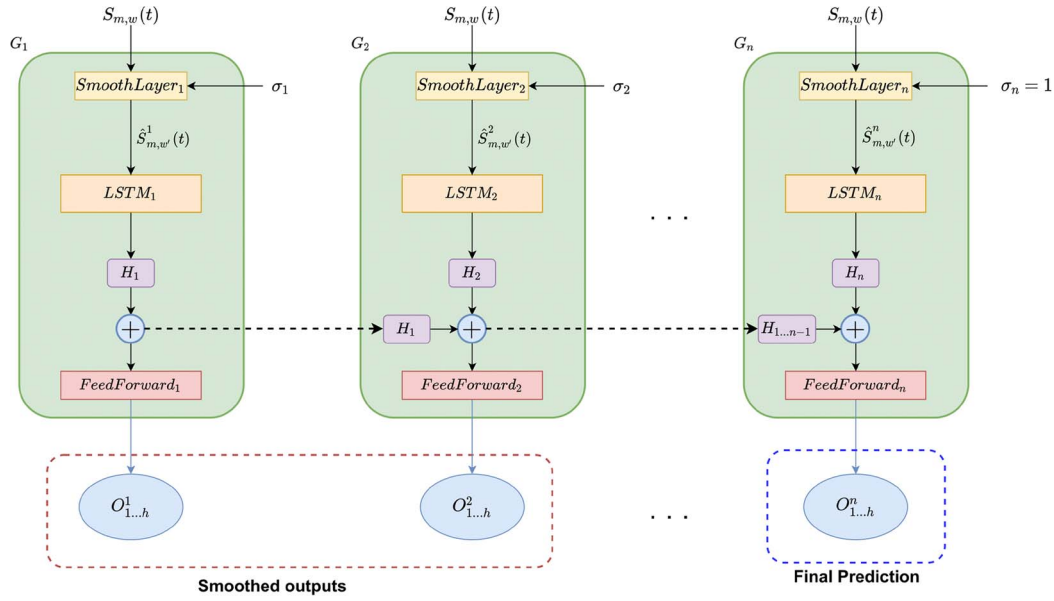


FIGURE 2. Proposed HLNet architecture.

During the training, the loss for each group is calculated using the second output and added to generate the final loss. Note that the prediction of each group must have the same smoothing process than the input using their corresponding σ.

4 Experimentation

4.1 Datasets

To validate our proposal, 13 publicly available datasets from different application fields have been selected. This collection of datasets represents a wide variety of time series forecasting problems, covering different domains, sequence/window sizes and forecasting horizons. Table 1 presents the characteristics of each dataset in detail. In total, the experiments involve more than 50,000 time series among all selected datasets. Furthermore, most of the selected datasets have been used in forecast competitions and other TSF reviews [10].

The Demand dataset [6] records the electricity demand from January 2015 to December 2015, which was included and obtained from the Spanish electricity network website. This dataset contains only one time series sampled from the same building every 10 minutes. We divided the time series into several non-overlapping segments of size 576 time steps, giving us 72 time series. The goal is to predict the values for the next 24 hours.

The CIF 2016 competition [27] dataset contains 72 monthly time series, of which 12 of them have a 6-month forecast horizon (CIF16o6) while the remaining 57 series have a 12-month forecast horizon (CIF16o12). Some of these time series are real bank risk analysis indicators while others are artificially generated.

Downloaded from https://academic.oup.com/jigpal/article/32/6/986/7670726 by Universidad de Burgos user on 11 September 2025

TABLE 1. Number of time series (N), forecast horizon (FH), maximum (M) and minimum (m) sequence sizes for each dataset used.

Dataset	N	FH	M	m
Demand	72	144	432	144
CIF16o12	57	12	36	15
CIF16o6	15	6	18	7
ExchangeRate	8	6	18	7
M3	1428	18	36	22
M4	48000	18	36	22
NN5	111	56	168	70
SolarEnergy	137	6	18	7
Tourism	336	24	48	30
Traffic	862	24	72	30
Traffic-metr-la	207	12	36	15
Traffic-perms-bay	325	12	24	15
WikiWebTraffic	997	59	118	73

The ExchangeRate (ER) dataset [16] records the daily exchange rates of 8 countries including Australia, UK, Canada, Switzerland, China, Japan, New Zealand and Singapore from 1990 to 2016 (7588 days for each country). The goal of this dataset is to predict values for the next 6 days.

M3 [22] and M4 [25] competitions include time series of different domains and observation frequencies. However, due to computational time constraints, we have limited the experimentation to the monthly time series as they are longer than the yearly, quarterly, weekly and daily time series. An 18-month prediction is required for both competitions and contains time series of different categories, such as industry, finance or demography. The number of instances belonging to each category is evenly distributed according to their presence in the real world, leading to representative conclusions. Taking into account only the monthly time series, the M4 dataset has 48000 time series, while M3 is considerably smaller with 1428 time series.

The NN5 competition dataset [4] is made up of 111 time series with a length of 735 values. This dataset represents 2 years of daily cash withdrawals at automatic teller machines (ATM) in England. The competition established a forecast horizon of 56 days.

The SolarEnergy (SE) [37] dataset contains solar power production records in 2006, which are sampled every 10 minutes from 137 solar photovoltaic power plants in the state of Alabama. The goal is to predict an hour.

The Tourism dataset [2] is made up of 336 monthly time series of different lengths, requiring a 24 months prediction. The data contain tourism bodies from Australia, Hong Kong and New Zealand, representing total tourism numbers at the country level.

Traffic-metr-la (TML) and Traffic-perms-bay (TPB) are two public traffic speed datasets [34]. Traffic-metr-la contains traffic speed information from 207 sensors on the highways of Los Angeles for 4 months. Similarly, Traffic-perms-bay records 6 months of statistics on traffic speed from 325 sensors in the Bay area. All data were aggregated every 5 minutes and the goal is to predict one hour in the future.

The Traffic dataset [31] is a collection of hourly time series from the California Department of Transportation collected during 2015 and 2016. The data describes the road occupancy rates

Downloaded from https://academic.oup.com/jigpal/article/32/6/986/7670726 by Universidad de Burgos user on 11 September 2025

(between 0 and 1) measured by different sensors on San Francisco Bay area freeways sampled every 10 minutes. The goal is to predict 4 hours in the future.

The WikiWebTraffic dataset (WWT) belongs to a Kaggle competition [9] with the goal of predicting future web traffic of a given set of Wikipedia pages. The web traffic is measured in the daily number of hits and the forecasting horizon is 59 days.

4.2 Experimental settings

The selected datasets contain different time series measured independently in different times and locations. For each time series in the datasets, the data is splitted into train, validation and test using a fixed origin way [10]. If each time series has length L, the window size is W and the horizon size is H. The size of the valid and test splits is $W + H$, while the train split is $L - W - H$. Note that the validation strategy selected evaluates a division for each time series and average the metrics for each one. This validation strategy provide a way of know the general performance of the model in all time series.

We set up a homogeneous experimentation environment for all models and datasets to ensure fair comparisons. A grid search has been used for hyperparameter search and the model with best results on the test set has been selected [29]. The hyperparameter space consists of hidden size, number of layers, sequence size and, just in our modified version, smooth factor σ. The hidden size has been selected to have 32, 64 or 128 neurons, the sequence size has been selected from a range depending of the forecasting horizon size using a 125%, 200% or 300% of the horizon size. The number of layers can be 2 or 4. Moreover, Adam optimizer was used for both models, and the training was stopped if there was no improvement in 5 consecutive epochs.

LSTM, DeepAR, NBEATS, TFT and ESN models has been selected to be compared with our proposal due to its remarkable results in several scenarios.

LSTM [11] model employs the original layer and uses the last hidden state to produce a multihorizon prediction.

DeepAR [26] uses a single LSTM layer to produce a normal distribution of a single-horizon forecasting. The mean of the distribution and an autoregressive approach has been selected to produce the final multihorizon forecasting and using an iterative approach.

NBEATS [24] model has been selected using the generic block proposed in the original paper, which does not rely on a seasonal-trend decomposition.

TFT [20] has been selected using a direct approach for the multi-horizon forecasting. As the model was not compatible with the libraries used during the experimentation, only the CPU was used for training and the number of layers was limited to 2.

ESN [12] employs another type of recurrent neural network obtaining competitive results compared with LSTM or GRU. As in LSTM, the last state is used to make the multihorizon forecasting.

All experiments have been executed under the same machine with the same hardware configuration. The hardware configuration is composed of an NVIDIA Geforce RTX 3070 GPU, an AMD Ryzen 7 5800X 3.8GHz and 32 GB RAM memory.

4.3 Evaluation metrics

We compared the results in the test instances between LSTM and HLNet with three forecasting metrics and one performance metric.

The weighted average percentage error (WAPE) was selected because it is more robust than the mean absolute percentage error for diverse datasets, when there exists values between 0 and 1. Additionally, in order to avoid a division by zero a constant $\epsilon = 1e^{-10}$ has been added into the

Downloaded from https://academic.oup.com/jigpal/article/32/6/986/7670726 by Universidad de Burgos user on 11 September 2025

TABLE 2. Best model parameters found by grid search.

	LSTM		DeepAR		NBEATS		TFT		ESN		HLNet		
	Layers	Units	Layers	Units	Layers	Units	Layers	Units	Layers	Units	Layers	Units	GF
Demand	2	32	1	64	2	128	2	64	2	128	4	64	2
CIF16o12	4	64	1	128	4	64	2	64	4	32	4	64	6
CIF16o6	4	32	1	128	2	64	2	64	4	64	2	64	2
ER	2	64	1	32	2	32	2	128	2	128	4	32	2
M3	2	128	1	64	2	32	2	64	4	32	2	32	6
M4	2	32	1	32	2	32	2	32	2	128	4	32	8
NN5	2	128	1	64	4	32	2	64	4	32	4	64	4
SE	2	32	1	128	4	64	2	128	4	64	4	32	2
Tourism	2	128	1	64	2	128	2	64	4	32	2	32	6
Traffic	2	128	1	128	4	64	2	128	2	32	2	64	4
TML	2	64	1	64	4	32	2	128	4	32	4	32	2
TPB	4	64	1	32	4	128	2	64	4	128	4	128	4
WWT	4	32	1	64	4	128	2	32	4	32	4	128	6

denominator.

$$WAPE = \frac{\sum_{t=1}^{h} |y_t - \hat{y}_t| * 100}{\epsilon + \sum_{t=1}^{h} y_t}$$

Finally, we measured the training time in order to show the performance of the model.

4.4 Results and discussion

Table 2 shows the best parameters by the grid search. In this table, we show the parameters in terms of layers, units for each layer and, just in our method, the σ parameter used for the SmoothLayer.

In general, there is no a clear pattern which relates the units and layers of the evaluated models. It seems that HLNet does not need more or less parameters than other models evaluated, not requiring more parameters in most of datasets.

Table 3 shows the efficacy of each model for all the datasets. In terms of efficacy, we may conclude that DeepAR and NBEATS performs poorly in most of datasets. We hypothesize that the autoregressive forecasting strategy in the DeepAR model and the decomposition learned by the generic block in NBEATS may be the cause of the low efficacy. The autoregressive may accumulate too much error that causes the error increases considerably and some datasets may not show a good seasonal or trend pattern. TFT and ESN models obtained remarkable results, but were not good enough to compare with LSTM and HLNet as they obtained better results in more datasets.

LSTM and HLNet show a similar efficacy in most of datasets. There are five datasets in which LSTM has better metrics and six datasets in which HLNet has better metrics. There are no notable differences in most datasets, but the Demand dataset shows at least a 10% improvement of HLNet over LSTM in all metrics.

[1] In terms of efficiency, Table 4 shows that DeepAR and NBEATS obtain the best results. However, as observed in Table 3 this improvement in efficiency comes with a considerable decrease

[1] Note than the training was performed by using a CPU.

Downloaded from https://academic.oup.com/jigpal/article/32/6/986/7670726 by Universidad de Burgos user on 11 September 2025

TABLE 3. WAPE (in percent) for each dataset evaluated.

	LSTM	DeepAR	NBEATS	TFT	ESN	HLNet
Demand	3.83	14.08	14.33	4.40	5.27	**3.27**
CIF16o12	17.20	23.99	15.86	14.64	**12.87**	15.32
CIF16o6	32.23	71.82	27.20	26.98	25.17	**19.93**
ER	0.30	13.02	0.36	0.39	0.32	**0.28**
M3	15.30	30.36	17.41	15.53	15.43	**15.28**
M4	**14.57**	41.73	15.73	15.07	14.75	14.78
NN5	18.56	41.90	41.29	18.51	18.89	**18.34**
SE	**222.00**	2255.52	300.56	233.96	259.96	255.00
Tourism	19.69	50.90	37.99	19.63	19.16	**18.95**
Traffic	**33.92**	78.45	72.84	39.37	34.67	34.40
TML	**3.33**	40.39	3.40	3.51	3.38	3.34
TPB	1.38	35.16	1.45	1.68	**1.36**	1.38
WWT	**46.89**	175.22	48.49	48.14	47.45	47.38

TABLE 4. Training time for each dataset and model evaluated.

	LSTM	DeepAR	NBEATS	TFT	ESN	HLNet
Demand	0’ 49”	8’ 02”	**0’ 11”**	50’ 32”	0’ 48”	3’ 40”
CIF16o12	0’ 30”	0’ 06”	**0’ 02”**	1’ 31”	1’ 07”	0’ 17”
CIF16o6	0’ 05”	0’ 01”	**0’ 01”**	0’ 21”	0’ 09”	0’ 04”
ER	2’ 05”	0’ 27”	**0’ 10’**	57’ 07”	10’ 31”	0’ 52”
M3	7’ 22”	1’ 20”	**0’ 11’**	56’ 51”	42’ 21”	2’ 49”
M4	9’ 15”	2’ 47”	**1’ 38”**	62’ 05”	137’ 49”	12’ 18”
NN5	20’ 09”	4’ 09”	**0’ 38”**	109’ 29”	146’ 24”	23’ 23”
SE	9’ 55”	7’ 16”	**1’ 56”**	42’ 35”	48’ 47”	7’ 21”
Tourism	3’ 41”	1’ 46”	**0’ 21”**	30’ 29”	32’ 33”	1’ 40”
Traffic	41’ 02”	6’ 42”	**1’ 42”**	73’ 16”	62’ 53”	21’ 12”
TML	14’ 14”	5’ 56”	**2’ 16”**	81’ 49”	172’ 58”	12’ 16”
TPB	48’ 27”	**2’ 28”**	5’ 23”	50’ 03”	84’ 22”	33’ 41”
WWT	46’ 49”	24’ 32”	**3’ 31”**	81’ 33”	345’ 27”	45’ 24”
Total	204’ 22”	65’ 32”	**18’ 00”**	696’ 37”	1086’ 09”	164’ 57”

in efficacy. For that reason, we do not consider these models in further discussions because they do not obtain good predictions for most of the problems.

Compared with LSTM, HLNet reduces the training time in 10 of 13 datasets. Additionally, in eight datasets, the reduction has been less than the half-time obtained by the LSTM. Note that this reduction in training time is not related to a reduction in the number of parameters, as shown in Table 2. This time reduction is achieved thanks to the isolation of the different groups in our method, which could allow us to improve the parallelization during the training.

4.5 Statistical test

In this section, a statistical test has been applied in order to make decision about the comparison among the results obtained by the different models. The statistical test selected is the Bayesian

Downloaded from https://academic.oup.com/jigpal/article/32/6/986/7670726 by Universidad de Burgos user on 11 September 2025

TABLE 5. WAPE and training time for every model and discretized horizon size.

	WAPE			Training time		
	Large	Medium	Short	Large	Medium	Short
HLNet	**10.93**	14.01	95.67	4’ 22”	8’ 57”	6’ 04”
DeepAR	100.06	44.46	537.41	12’ 47”	4’ 46”	1’ 53”
ESN	19.07	18.16	95.14	72’ 06”	48’ 25”	27’ 19”
LSTM	13.49	14.62	**94.16**	10’ 27”	15’ 33”	9’ 32”
NBEATS	31.67	23.69	99.78	**1’ 15”**	**1’ 19”**	**0’ 50”**
TFT	33.24	**13.3**	116.14	66’ 58”	43’ 22”	44’ 20”

version of Wilcoxon signed-rank test that uses a Dirichlet process as prior information. As the original Wilcoxon signed-rank test, the method receives a pair of values from two models (A and B) and a range of statistical equivalence. Values correspond to metrics obtained after the evaluation process in each dataset. The test outputs the probabilities that A has higher values than B, that B has higher values than A and equivalent metrics (also called rope) in the posterior distribution.

The only hyperparameter needed is the range of statistical equivalence. As we consider the WAPE and Training time as the two main metrics to apply the test, we must define two ranges. For WAPE, one percent have been established, which means that metrics with less than one percent of difference are considered equivalent. For the training time, a minute has been selected, which means that models with a difference of less than a minute are considered equivalent.

Figure 3 shows the results obtained after applying the test to the results obtained in Table 3. The results are split into two columns, each figure representing the WAPE. Each figure represents a pyramid with the probabilities from HLNet and the other three proposed methods after comparing their corresponding metrics. For instance, the p(HLNet), p(rope), p(DeepAR) corner represents the probability of HLNet having greater metric than DeepAR, DeepAR having greater metric than HLNet and probability of no relevant difference.

In terms of WAPE, we can conclude than HLNet has lower error than DeepAR, NBEATS and ESN as observed in Figure 3a, Figure 3c and Figure 3e. However, Figure 3b shows that LSTM and HLNet have a similar WAPE in all datasets with a 95.6% of probability. Finally, Figure 3d shows inconclusive results, as the percentage was not high enough.

In terms of Training time, we can conclude than HLNet slower than DeepAR and NBEATS as observed in Figure 4a and Figure 4d. However, Figure 4b, Figure 4d and Figure 4e show that LSTM, TFT and ESN have a longer training time with a probability of 96.7%, 100% and 100%, respectively.

Finally, HLNet seems to be the fastest method, supported by the statistical test and Table 3, while it does not considerably decrease efficacy.

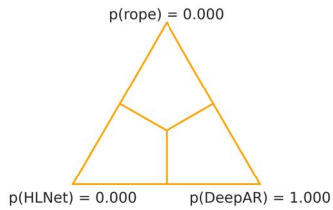
4.6 Horizon length analysis

In this section, we analysed the impact of horizon length in the efficacy and efficiency over the different model. The horizon size of every dataset has been discretized into large, medium and short. Large corresponds to horizons with more than 50 instances, medium corresponds to horizons between 20 and 50 instances and short corresponds to horizons with less than 20 instances.

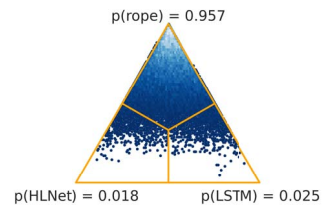
Table 5 shows the average WAPE and training time for each model and discretized horizon size.

HLNet seems to be the best option for large horizon sizes followed by LSTM. It obtained the second best training time after NBEATS but with a remarkable improvement in WAPE. In general, the method seems to be stable in terms of efficacy and efficiency without great flaws.

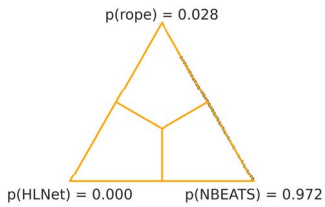
Downloaded from https://academic.oup.com/jigpal/article/32/6/986/7670726 by Universidad de Burgos user on 11 September 2025



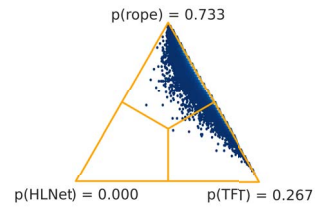
(a) DeepAR.



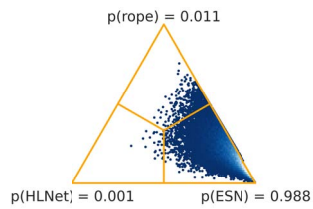
(b) LSTM.



(c) NBEATS.



(d) TFT.



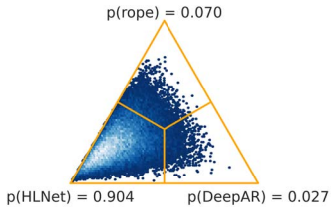
(e) ESN.

FIGURE 3. The posterior distribution of HLNet has a higher WAPE value than those of the compared models.

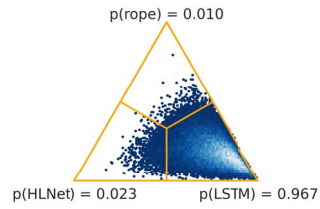
TFT obtained the best results for medium horizon sizes followed by HLNet. However, the training time is the second worst only surpassed by ESN. For that reason, HLnet would be a better option with a similar efficacy and considerably shorter training time. For large and short horizon sizes, the error is significantly greater than other models, which seems to show an unstable behavior independently of the horizon size.

LSTM obtained the best results for short horizon sizes followed by ESN. In terms of training time was considerably better than ESN, which makes LSTM a better option. However, the training time

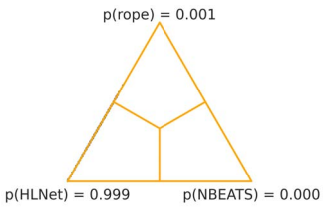
Downloaded from https://academic.oup.com/jigpal/article/32/6/986/7670726 by Universidad de Burgos user on 11 September 2025



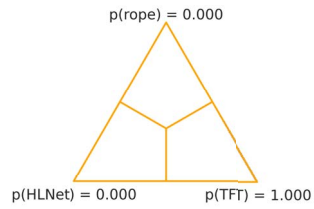
(a) DeepAR.



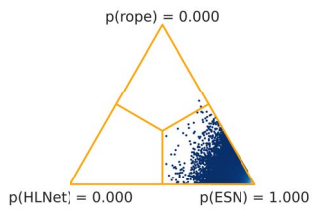
(b) LSTM.



(c) NBEATS.



(d) TFT.



(e) ESN.

FIGURE 4. The posterior distribution of HLNet has a longer training time than the compared models.

was still longer than NBEATS. In general, LSTM obtains a stable and good efficacy for all horizon sizes.

4.7 Analysis of the number of time series

In this section, an analysis of the impact in the number of time series available in each dataset is performed. The number of time series has been discretized into three groups depending on the quantity of time series: high, medium and low. The datasets with more than 500 different time series

Downloaded from https://academic.oup.com/jigpal/article/32/6/986/7670726 by Universidad de Burgos user on 11 September 2025

TABLE 6. WAPE and training time for every model and number of time series group.

	WAPE			Training time		
	High	Medium	Low	High	Medium	Low
HLNet	**26.11**	**93.08**	**10.82**	16’ 13”	9’ 56”	0’ 52”
DeepAR	86.74	542.14	30.01	11’ 03”	4’ 18”	0’ 54”
ESN	32.11	113.8	12.34	89’ 38”	68’ 07”	9’ 12”
LSTM	27.56	100.57	12.79	35’ 01”	16’ 18”	1’ 18”
NBEATS	40.42	100.37	14.54	**1’ 49”**	**1’ 34”**	**0’ 10”**
TFT	30.75	95.67	19.64	71’ 08”	58’ 30”	20’ 40”

TABLE 7. WAPE and training time for all models and frequencies.

	WAPE			Training time		
	Daily	Minutely	Monthly	Daily	Minutely	Monthly
HLNet	**89.78**	**18.49**	13.83	9’ 8”	2’ 51”	8’ 60”
DeepAR	490.21	44.58	73.09	5’ 38”	2’ 08”	7’ 47”
ESN	98.62	20.62	15.46	56’ 00”	19’ 17”	65’ 41”
LSTM	96.68	20.71	**13.55**	15’ 04”	4’ 35”	17’ 42”
NBEATS	99.63	22.79	30.41	**1’ 48”**	**0’ 28”**	**1’ 07”**
TFT	101.51	22.81	22.87	43’ 13”	24’ 34”	89’ 54”

are in the high group, data sets whose number of time series is between 300 and 1000 are in the medium group and datasets with less than 300 time series are in the low group.

Table 6 shows the results obtained after grouping and average the the results obtained by the number of time series in each dataset. As expected, the time required for datasets with a high amount of time series are usually greater than for datasets with a few time series.

In general, HLNet obtained the best results in terms of efficacy though obtaining worst performance than NBEATS. However, HLNet obtained the third best model for high and medium number of time series and the second for low number of time series.

ESN, LSTM and TFT obtained a good efficacy compared with HLNet. However, the efficiency is remarkably worst, which supports our idea that HLNet is the best option in general.

4.8 Frequency analysis

In this section, we averaged the results by the sample frequency of the time series in each dataset. The datasets contain time series samples in three frequencies: daily, minutely and monthly.

Table 7 shows the efficacy and efficiency obtained for each model and frequency. As observed, datasets sampled using a daily frequency have higher errors than minutely and monthly frequencies.

For daily and minutely frequencies, HLNet obtained the best efficacy with by great margin with the second best models (LSTM and ESN). In terms of efficiency, HLNet is the third best option surpassed by DeepAR and NBEATS.

For monthly frequency, LSTM obtained the best efficacy followed by HLNet. However, the training time is almost twice the obtained by the HLNet architecture with little improvement in efficacy.

Downloaded from https://academic.oup.com/jigpal/article/32/6/986/7670726 by Universidad de Burgos user on 11 September 2025

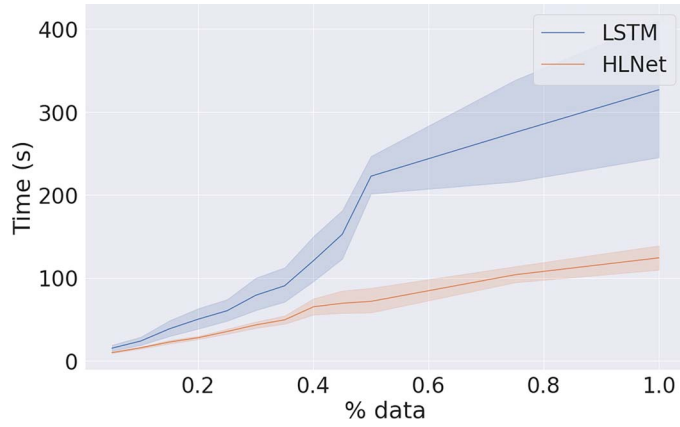


FIGURE 5. Training time analysis by data size in M3 dataset.

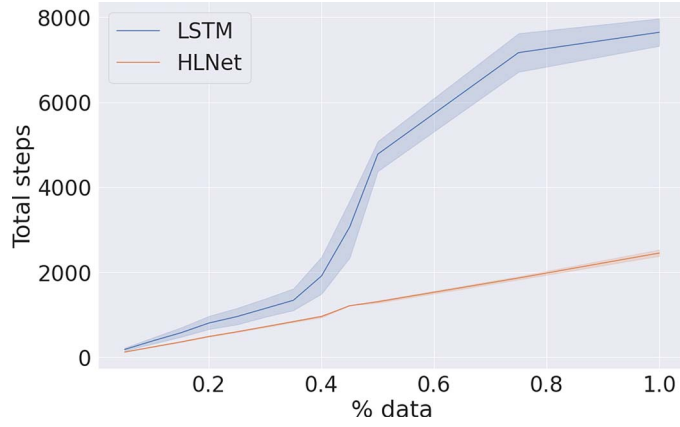


FIGURE 6. Total train steps by data size in the M3 dataset.

4.9 Data size analysis

We used M3 dataset to study the evolution of the training time over the data size as the training time is not too high and the training time reduction is good enough to see the difference between HLNet and LSTM. The data has been sampled from 5% to 100% of available data from more recent to older events. We used this sampling method because we consider the most realistic scenario in practice. Additionally, the sampling step is 5% until the 50% of the total data size and then the sampling rate is increased to 25% after the 50% of the total data size.

Figure 5 shows how the training time scales in our method as more data are added to our method. We can observe how the methods start to have the same training time at the beginning, but the growth of the execution time of LSTM is much greater than that of HLnet, especially since the point of 0.5 (50%) of data size.

The training time growth in LSTM supports the hypothesis that HLNet is able to converge faster needing to process a lower number of batches. This means that as the simpler tasks converge faster

Downloaded from https://academic.oup.com/jigpal/article/32/6/986/7670726 by Universidad de Burgos user on 11 September 2025

by creating general representations of the series and transfer their knowledge to the next layers, the following layers can learn a more detailed representation faster. On the contrary, LSTM needs extra time to learn the general representation of the series. These results emphasize the idea that recurrent neural networks organize their knowledge in a hierarchical way, from more general representations to more specific ones.

5 Conclusions and future works

In this paper we have introduced HLNet, a novel method for time series multistep forecasting. Thus, we have added a structural change in LSTM networks that allows learning from simpler to more complex time series in a hierarchical way. This is done by smoothing the original input and transferring the extracted knowledge to the subsequent levels. The results have shown a great improvement in terms of training time keeping the efficacy, which leads to the use of deeper models with less execution time. As future works, other smoothing strategies rather than MA could be followed. Additionally, the development of a custom loss function with different weights for each level and evolving with the number of epochs is proposed.

Acknowledgements

This research has been supported by the grant PID2020-117954RB-C22 and PID2020-117954RB-C21 funded by MICIU/AEI/ 10.13039/501100011033. This work has also been supported by TED2021-131311B-C21 and TED2021-131311B-C22 funded by MICIU/AEI/10.13039/501100011033 and by the European Union NextGenerationEU/PRTR and Junta de Andalucía for projects PYC20 RE 078 US, PY20-00870 and UPO-138516. Funding for open access publishing: Universidad de Sevilla's library.

References

[1] D. Afolabi, S. Guan, K. Man, P. W. H. Wong and X. Zhao. Hierarchical meta-learning in time series forecasting for improved interference-less machine learning. *Symmetry*, **9**, 283, 2017.

[2] G. Athanasopoulos, R. J. Hyndman, H. Song and D. C. Wu. The tourism forecasting competition. *International Journal of Forecasting*, **27**, 822–844, 2011.

[3] F. Bahrpeyma, A. Mccarren and M. Roantree. Multi-resolution forecast aggregation for time series in Agri datasets. In *Proceedings of the Irish Conference on Artificial Intelligence and Cognitive*, vol. 25, pp. 1–12, 2017.

[4] S. F. Crone. *NN5 Time Series Forecasting Competition for Neural Networks*, http://www.neural-forecasting-competition.com/, 2008.

[5] D. Jinliang, C. Xiusi, J. Renhe, S. Xuan and W. T. Ivor. A Multi-View Multi-Task Learning Framework for Multi-Variate Time Series Forecasting. In *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, pp. 7665–7680, 2023.

[6] Red Eléctrica de España www.ree.es, 2015.

[7] A. Galicia, R. Talavera-Llames, A. Troncoso, I. Koprinska and F. Martínez-Álvarez. Multi-step forecasting for big data time series based on ensemble learning. *Knowledge-Based Systems*, **163**, 830–841, 2019.

[8] A. Galicia, J. F. Torres, F. Martínez-Álvarez and A. Troncoso. A novel spark-based multi-step forecasting algorithm for big data time series. *Information Sciences*, **467**, 800–818, 2018.

Downloaded from https://academic.oup.com/jigpal/article/32/6/986/7670726 by Universidad de Burgos user on 11 September 2025

[9] Google *Web Traffic Time Series Forecasting Competition*, 2017. https://www.kaggle.com/c/web-traffic-time-series-forecasting
[10] C. Bergmeir, H. Hewamalage and K. Bandara. Recurrent neural networks for time series forecasting: current status and future directions. *International Journal of Forecasting*, **37**, 388–427, 2021.
[11] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, **9**, 1735–1780, 1997.
[12] H. Jaeger. *The Echo State Approach to Analysing and Training Recurrent Neural Networks-with an Erratum Note*, vol. 148. German National Research Center for Information Technology GMD. Technical Report, Bonn, Germany, 2001.
[13] A. Koenecke and A. Gajewar. Curriculum learning in deep neural networks for financial forecasting. In *Lecture Notes in Computer Science*, p. 11985, 2020.
[14] N. Kourentze, F. Petropoulos and J. R. Trapero. Improving forecasting by estimating time series structural components across multiple frequencies. *International Journal of Forecasting*, **30**, 291–302, 2014.
[15] N. Kourentzes and F. Petropoulos. Forecasting with multivariate temporal aggregation: The case of promotional modelling. *International Journal of Production Economics*, **181**, 145–153, 2016.
[16] G. Lai, W. Chang, Y. Yang and H. Liu. Modeling long- and short-term temporal patterns with deep neural networks. In *Proceedings of the International ACM SIGIR Conference on Research and Development in Information Retrieval*, vol. 41, pp. 95–104, 2018.
[17] P. Lara-Benítez, M. Carranza-García and J. C. Riquelme. An experimental review on deep learning for time series forecasting. *International Journal of Neural Systems*, **31**, 2130001, 2021.
[18] S. Li, X. Jin, Y. Xuan, X. Zhou, W. Chen, Y. Wang and X. Yan. *Enhancing the Locality and Breaking the Memory Bottleneck of Transformer on Time Series Forecasting*, 2019.
[19] L. Liebel and M. Körner. *Auxiliary Tasks in Multi-Task Learning*, arXiv, 2018.
[20] B. Lim, S. O. Arik, N. Loeff and T. Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, **37**, 1748–1764, 2021.
[21] T. Lin, B. G. Horne, P. Tino and C. L. Giles. Learning long-term dependencies in NARX recurrent neural networks. *IEEE Transactions on Neural Networks*, **7**, 1329–1338, 1996.
[22] S. Makridakis and M. Hibon. The M3-competition: results, conclusions and implications. *International Journal of Forecasting*, **16**, 451–476, 2000.
[23] F. Martínez-Álvarez, A. Troncoso, G. Asencio-Cortés and J. C. Riquelme. A survey on data mining techniques applied to electricity-related time series forecasting. *Energies*, **8**, 13162–13193, 2015.
[24] B. N. Oreshkin, D. Carpov, N. Chapados and Y. Bengio. *N-BEATS: Neural Basis Expansion Analysis for Interpretable Time Series Forecasting*. arXiv, 2019.
[25] E. Spiliotis, S. Makridakis and V. Assimakopoulos. The M4 competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting*, **36**, 54–74, 2020.
[26] D. Salinas, V. Flunkert, J. Gasthaus and T. Januschowski. DeepAR: probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, **36**, 1181–1191, 2020.
[27] M. Stepnicka and M. Burda. *Computational Intelligence in Forecasting (CIF)*, https://irafm.osu.cz/cif/main.php, 2016.

Downloaded from https://academic.oup.com/jigpal/article/32/6/986/7670726 by Universidad de Burgos user on 11 September 2025

[28] R. Talavera-Llames, R. Pérez-Chacón, A. Troncoso and F. Martínez-Álvarez. MV-kWNN: a novel multivariate and multi-output weighted nearest neighbours algorithm for big data time series forecasting. *Neurocomputing*, **353**, 56–73, 2019.

[29] J. F. Torres, A. Galicia, A. Troncoso and F. Martínez-Álvarez. A scalable approach based on deep learning for big data time series forecasting. *Integrated Computer-Aided Engineering*, **25**, 335–348, 2018.

[30] J. F. Torres, D. Hadjout, A. Sebaa, F. Martínez-Álvarez and A. Troncoso. Deep learning for time series forecasting: a survey. *Big Data*, **9**, 3–21, 2021.

[31] https://pems.dot.ca.gov/ *California Department of Transportation*, 2015.

[32] A. van den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior and K. Kavukcuoglu. *WaveNet: A Generative Model for Raw Audio*. arXiv, 2016.

[33] X. Wu, B. Shi, Y. Dong, C. Huang, L. Faust and N. V. Chawla. RESTFul: resolution-aware forecasting of behavioral time series data. In *Proceedings of the ACM International Conference on Information and Knowledge Management*, pp. 1073–1082, 2018.

[34] C. Shahabi, Y. Li, R. Yu and Y. Liu. Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting. In *Proceedings of the International Conference on Learning Representations*, vol. 18, 2018.

[35] M. D. Zeiler and R. Fergus. Visualizing and understanding convolutional networks. In *Proceeddings of the European Conference on Computer Vision*, vol. 8689, pp. 818–833, 2014.

[36] A. Zeng, M. Chen, L. Zhang and Q. Xu. Are Transformers Effective for Time Series Forecasting? In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 23, 2022.

[37] Y. Zhang. *Solar Power Data for Integration Studies (NREL)*, https://www.nrel.gov/grid/solar-integration-data.html, 2007.

[38] C. Zheng, X. Fan, C. Wang and J. Qi. GMAN: a graph multi-attention network for traffic prediction. In *Gman: A Graph Multi-Attention Network for Traffic Prediction*, vol. 34, pp. 1234–1241, 2019.

[39] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong and W. Zhang. Informer: beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, pp. 11106–11115, 2021.

Received 20 May 2022

Downloaded from https://academic.oup.com/jigpal/article/32/6/986/7670726 by Universidad de Burgos user on 11 September 2025